Recoverable Releases Across Package Registries: A Git-Backed Saga Protocol

Semen Fediukovich
Independent Researcher
yohimik@gmail.com

ABSTRACT

Releasing interdependent packages writes irreversibly to independent registries without a coordinator or rollback. A partial failure can therefore leave a consumer permanently behind a dependency that was published.

This paper presents CCME, a commit-message language and release protocol for resumable releases, and dispat, its open-source implementation for single-package repositories, monorepos, and polyrepos joined through a control repository. In the formal model, release state consists of git tags, the plan is a pure function of repository history, dependency graph and configuration, and propagated work is admitted against the consumer's pending window. We prove eight guarantees, including no orphans, exactly-once ledger discharge and convergence under an explicit retry invariant. Minimal counterexamples establish why delivery must be tracked per consumer, why prerelease aggregation needs a separate window, and why retry guarantees require stable channel and suppression predicates.

Against the official dispat 1.8.2 image, the fresh campaign passes six selected CCME 2.0 regressions, agrees with a restricted reference oracle on all 1,500 core instances, recovers 24/24 injected-failure episodes within four runs, and completes eight scale cells. The full profile intentionally generates regressive prereleases: 997 plans were accepted and 503 complete diagnostic plans were rejected with E195. All 1,914 differing rows retain their heuristic classification and stated coverage limits.

CCS CONCEPTS

• **Software and its engineering → Software configuration management and version control systems**; • **Information systems** → *Database transaction processing*.

KEYWORDS

release engineering, monorepo, polyrepo, semantic versioning, conventional commits, sagas, partial failure, idempotency, deterministic replay

1 INTRODUCTION

A release workspace can contain one package, several packages in a monorepo, or packages drawn from separately maintained repositories (a polyrepo). Releasing it means choosing versions for the packages that need them, writing those versions into manifests, building artefacts, uploading them to registries, and recording what happened. In a modern project the registries are plural and heterogeneous: an npm package sits next to a Go module, a Docker image, and a documentation site, all wired into one dependency graph.

The recording is the hard part. Independent registries offer no atomic transaction spanning the release: each publish leg may succeed or fail, and the outcome may be lost before the corresponding git tag is written. This is a long-running transaction over separately committed resources, the setting for sagas [1]; recovery must complete or reconcile successful legs rather than assume a cross-registry rollback.

Existing tools expose different ledgers and recovery boundaries. Lerna, Nx [18], release-please, and Changesets [16] automate releases, while semantic-release [17] targets one package per repository. Their pinned fault experiments below show that registry queries, intent files, and tag timing answer different parts of what an interrupted cross-registry run still owes.

This paper treats the release as the transaction it is, and makes three contributions:

- a commit-message language, CCME, in which release intent (blast radius on two separate, one-way-coupled axes, version bumps and prerelease channels) is reviewable text in the commit (Section 3);
- a formal model of the release computation and publish protocol, self-contained in this paper, with eight guarantees proved over it (Sections 4 and 5); minimal counterexamples identify necessary design choices and the explicit invariant under which retries preserve the guarantees (Section 8.3);
- dispat, an ecosystem-agnostic implementation in which a package is a directory and a stage is a shell command, evaluated by differential validation, selected CCME 2.0 vectors, recovery campaigns, replay, and pinned fault experiments (Sections 7 and 8).

2 MOTIVATING EXAMPLE

A workspace holds a library `core` and three consumers `cli`, `ui`, `api`, each depending on `core`. A commit C lands with the message `feat(core)^: streaming reader`: `core` takes a minor bump, and the caret propagates a patch bump one edge outward. The run publishes `core@1.5.0`, `ui@0.9.2` and `api@1.2.1`, tagging each on success, and then `cli`'s publish fails, for any of the reasons registry writes fail.

The natural expectation is that re-running finishes the job. Whether it does depends on one definition. Suppose the engine keeps a commit's release intent alive only while the commit's *own* package still has it pending, the natural reading. Then on the second run `core` has released C, so the unit has no source packages, propagates to nothing, and `cli` is never released, on this or any future run. `cli` is *orphaned*: permanently one patch behind a dependency it declares, with no diagnostic, because from the engine's point of view nothing is pending. The failure is silent, and grows more likely with the fan-out.

Table 1: Header examples.

Header	Meaning
`feat: add retry`	derived scope, minor bump
`fix(api): null guard`	`api`, patch bump
`perf(core)^patch+1: x`	patch to direct consumers
`refactor(core)^^major!: x`	breaking; major to closure
`feat(cli)%beta: watch mode`	`cli` onto the beta line
`feat(core)^%beta++1: x`	self beta; consumers follow
`release(a)%beta>stable: ship`	graduate those still on beta
`cancel(*): reset state`	discard pending records

The repair is a one-line change of definition: a unit propagates to a dependent exactly while the *dependent's* pending window still contains the unit's commit (Definition 4.10). Catch-up then needs no repair pass or side state when prior outcomes are confirmed by tags; an uncertain external publish/tag gap still requires identity reconciliation. Section 5 proves the guarantee this buys (G2, no orphans) and its siblings.

3 CCME: THE LANGUAGE

CCME is a strict superset of Conventional Commits 1.0.0 [7] over a stated conforming subset. One commit message is a sequence of *units* separated by a whole-line separator (`---`), each an independent conventional-commit-shaped block with its own header, body and footers, so a squash-merged pull request describes several unrelated changes without one lie of a type for all of them:

```
feat(@acme/core)^^minor%beta++*: reader
---
fix(@acme/cli): correct exit code on SIGINT
---
cancel(@acme/legacy-adapter): reset state
```

Scopes name packages, or are derived. The scope is a comma-separated package set, globs with exclusions (`feat(*,-docs)`), or absent, in which case the affected set is derived from the files the commit changed, package by owning directory. Derivation makes ordinary unscoped commits release the packages they touched.

Two axes, two depths. A commit answers two separate questions about its dependents: how far does the *version bump* travel, and how far does the *prerelease channel* travel. Both default to depth zero: a commit that says nothing releases its own packages and nothing else. The bump axis is written `^` (direct consumers), `^^` (transitive closure) or `+N`; the channel axis `%` (own packages change channel), `%%` (dependents do), `++N` (how far). Every sigil desugars to a footer; the header above desugars to

```
feat(@acme/core): reader

Propagate: minor          Channel: beta
Propagate-Depth: all      Propagate-Channel-Depth: all
```

reading: `core` takes a minor bump and enters the beta line, every transitive dependent takes a minor bump and follows it onto `beta`. Channel values may be *transitions* `<from>><to>`: move only packages currently on `<from>`. Transitions make graduation idempotent: `release(@acme/*)%beta>stable: ship` graduates whatever is still on beta and can be replayed safely after a partial failure.

Cancellation and corrections. The type `cancel` discards a scope's accumulated unreleased release metadata; a unit is cancelled for a package when its commit is an ancestor-or-self of the cancelling commit; ancestry, unlike timestamps, is deterministic under merges and rebases. The footers `Edits: <sha>` and `Deletes: <sha>` restate or discard a named past record: a commit `fix(core): rewrite internals` carrying `Edits: <sha of A>` replaces A's pending major with this unit's patch, so `core` releases `1.4.3` rather than `2.0.0`. Corrections reach only unreleased work and only strict ancestors; the newest correction wins; a correction whose own record is discarded is void. `Release-As: none` holds a package until lifted; an exact `Release-As` pins one version.

4 THE MODEL

This section defines the release computation and the publish protocol; Section 5 proves the guarantees over these definitions and nothing else.

4.1 Workspace, State, Windows

Definition 4.1 (history). The history is a finite DAG of commits with a distinguished $\mathsf{HEAD} = h$; $reach(c)$ is the set of commits reachable from c by parent edges, including c; write $c' \preceq c$ for $c' \in reach(c)$. h is fixed throughout. This reading of a repository is the standard one: the commit DAG with reachability is the working model of git history in repository mining [9], and the same structure underlies formal derivations of git's three-way merge from least common ancestors [10]. Unlike the patch-theoretic accounts of version control [11–13], this model tracks no file contents at all, only history, tags and messages, which is what makes the plan a pure function of the repository.

Definition 4.2 (workspace). $\mathcal{P}$ is the finite set of packages at h; $E \subseteq \mathcal{P} \times \mathcal{P}$ contains (a, b) when a declares a dependency on b of a configured kind; $dep(p) = \{d : (d, p) \in E\}$. The graph is acyclic (a cycle is a repository-scoped error: no publish order exists for it, so whichever member went first would declare a range on a version that does not exist). For $S \subseteq \mathcal{P}$, $\delta(S, d)$ is the least number of edges on a chain from a member of S to d against the arrows, ∞ if none.

Definition 4.3 (bumps, versions, channels). $\mathbb{B} = \{\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}\}$ is a chain, hence a join-semilattice under $\sqcup = \max$. Versions are SemVer 2.0.0 [6] with its precedence; $ch(v)$ is stable for a non-prerelease v and otherwise v's first prerelease identifier; $core(v)$ and $cnt(v)$ are the numeric triple and the prerelease counter; $apply(v, b)$ applies a bump.

Definition 4.4 (state). A state σ is a finite set of release tags (p, v, c) with $c \preceq h$; states are ordered by inclusion. $base_\sigma(p)$ is the highest-precedence version among p's tags, $sbase_\sigma(p)$ the highest-precedence *non-prerelease* one, $basec_\sigma(p)$ and $scom_\sigma(p)$ their commits; $ch(base_\sigma(p)) = \mathsf{stable}$ by convention when p has no tag. Tags are never moved or deleted, so states only grow: a tag is the record that a leg of the transaction completed, and a rewritten record is what makes a saga unrecoverable. Tag write access is therefore release authority; a manifest's version field is not the release ledger. Manifests, configuration, and derived scopes remain trusted planning inputs.

Definition 4.5 (train and fresh-work windows).

$$W_\sigma(P) \;=\; reach(h) \setminus reach(scom_\sigma(P)), \tag{1}$$

and

$$W^{\circ}_{\sigma}(P) = W_{\sigma}(P) \setminus reach(basec_{\sigma}(P)), \tag{2}$$

with an empty subtracted reach when P has no tag. W_{σ} is the *train window*; W°_{σ} is the *fresh-work window*. When P has no stable baseline, $W_{\sigma}(P) = reach(h)$: an unreleased package's first release accounts for its whole history, which is also why adoption on an existing repository needs no migration step. The window is measured from the last *stable* tag so that a prerelease train can recompute its target over the whole train (Eq. 8). Direct admission, propagation admission and ledger discharge use W° instead. Thus a prerelease tag discharges delivered work without erasing the earlier records needed to recompute the train target.

4.2 Units, Suppression, Admission

Definition 4.6 (units). $\mathcal{U}$ is the set of units carried by commits in $\bigcup_P W_{\sigma}(P)$, parsed and with scopes resolved and corrections applied. For $u \in \mathcal{U}$: c_u is its commit, $scope(u) \subseteq \mathcal{P}$ its resolved scope, $bump(u) \in \mathbb{B}$ the bump its type and `!` map to; its bump axis is $pb(u) \in \mathbb{B}$ (the stated value, or $bump(u)$ under `inherit`; the default once propagation is asked for is `patch`, the honest signal for a dependent whose own API did not change), $pd(u) \in \mathbb{N} \cup \{\infty\}$ (default 0), $ps(u)$ a scope filter; its channel axis is a channel value $pc(u)$, a depth $pcd(u)$ (default 0) and a filter $pcs(u)$.

Definition 4.7 (suppression and sources). $canc(c, p)$ holds when c is an ancestor-or-self of a `cancel` commit whose scope contains p; $held(p)$ when p's effective `Release-As` is `none`. Both are functions of h and the configuration alone. With $disch_{\sigma}(p, c) \equiv c \notin W^{\circ}_{\sigma}(p)$,

$$src_{\sigma}(u) = \{p \in scope(u) : disch_{\sigma}(p, c_u) \vee \neg(canc(c_u, p) \vee held(p))\}. \tag{3}$$

Suppression applies only to *undischarged* work: once p has published the version carrying u, the artefact its consumers are owed is public, and neither a later cancel nor a later hold may retroactively strand them.

Definition 4.8 (reach). $R(S, \ell) = \{d \in \mathcal{P} \setminus S : \delta(S, d) \leq \ell\}$. Excluding all of S means a unit never propagates to a package it bumps directly; depth is shortest-path and always measured from the originating source set, and a propagated effect never cascades its own propagation, so the depth written in the message is the whole blast radius in every run.

Definition 4.9 (channels). $chan_{\sigma}(P)$, resolved for *every* package: the value proposed by the newest direct `Channel` directive naming P with $c_u \in W^{\circ}_{\sigma}(P)$; else the value proposed for P by the newest channel-axis unit u with $pcd(u) > 0$, $P \in R(src_{\sigma}(u), pcd(u)) \cap pcs(u)$, $c_u \in W^{\circ}_{\sigma}(P)$ and $\neg canc(c_u, P)$; else $ch(base_{\sigma}(P))$. A proposal is computed against $cur = ch(base_{\sigma}(P))$ (the tag, never a value computed this run) and is empty when it proposes *cur* itself; a transition proposes only where *cur* matches its `<from>`; a *propagated* non-transition `stable` never graduates a prerelease package (graduation must be deliberate); `inherit` resolves to the unit's own `Channel` value if any, else to $ch(base_{\sigma}(q))$ for its sources q, bytewise least name deciding a disagreement.

Definition 4.10 (admission and effective bump). Let $live(u)$ mean $bump(u) \neq \mathsf{none}$, $pb(u) \neq \mathsf{none}$, and $pd(u) > 0$. With $srcCh_{\sigma}(u)$ equal to $\{chan_{\sigma}(p) : p \in src_{\sigma}(u)\}$ and

$$rsv_{\sigma}(u, D) \equiv \mathsf{stable} \in srcCh_{\sigma}(u) \vee chan_{\sigma}(D) \in srcCh_{\sigma}(u) \tag{4}$$

(an installer selects a prerelease only on its own line, so a bump across a channel boundary is not a real obligation),

$$\begin{aligned} adm_{\sigma}(u, D) \;\equiv\; & D \in R(src_{\sigma}(u), pd(u)) && \text{(reached)} \\ \wedge\; & D \in ps(u) && \text{(in scope)} \\ \wedge\; & c_u \in W^{\circ}_{\sigma}(D) && (\textbf{consumer's} \text{ window}) \\ \wedge\; & \neg\, canc(c_u, D) && \text{(not discarded)} \\ \wedge\; & live(u) && \text{(bump-live)} \\ \wedge\; & rsv_{\sigma}(u, D) && \text{(resolvable),} \end{aligned} \tag{5}$$

$$eff_{\sigma}(P) = dir_{\sigma}(P) \sqcup \bigsqcup \{\, pb(u) : adm_{\sigma}(u, P) \,\}, \tag{6}$$

where $dir_{\sigma}(P) = \bigsqcup\{bump(u) : P \in src_{\sigma}(u) \cap scope(u),\ c_u \in W^{\circ}_{\sigma}(P)\}$. Because $\sqcup$ is a join, a package with its own `feat` pending is unaffected by an incoming propagated `patch`, propagation can only raise a version, and gathering order cannot matter.

The consumer-side staleness audit is the upward dual of Eq. 5, not merely reverse reachability. For candidate D it scans units in $W^{\circ}_{\sigma}(D)$, excludes D when it is itself a source of that unit, and applies the same shortest depth, scope, cancellation, *live*, and resolved-channel predicate *rsv*. These clauses make the upward answer exactly equal to the downward planner's obligations, including prerelease workspaces.

4.3 Plan, Versions, Runs

Definition 4.11 (plan and versions). Write $pin_{\sigma}(P) = v$ when the effective surviving exact `Release-As: v` unit for P has $c_u \in W^{\circ}_{\sigma}(P)$; otherwise $pin_{\sigma}(P)$ is undefined. Thus a successful tag discharges a pin just like other fresh work, and the directive is fixed during a retry.

$$\begin{aligned} \Pi(\sigma) = \{\, P : \neg held(P) \wedge (\, & eff_{\sigma}(P) \neq \mathsf{none} \\ & \vee\ chan_{\sigma}(P) \neq ch(base_{\sigma}(P)) \ \vee\ pin_{\sigma}(P)\downarrow) \,\}, \end{aligned} \tag{7}$$

each member carrying $v_{\sigma}(P)$. Let $S = sbase_{\sigma}(P)$, $B = base_{\sigma}(P)$, and E be the train aggregate over $W_{\sigma}(P)$. If $pin_{\sigma}(P)\downarrow$, it supplies that exact version. A stable package that remains stable uses $apply(S, eff_{\sigma}(P))$; a prerelease package graduating to stable uses $apply(S, E)$ as required by CCME 2.0 §11.5. Otherwise, with *ch* the target channel,

$$t = apply(S, E),$$

$$v_{\sigma}(P) = \begin{cases} t\text{-}ch.(cnt(B)+1) & \text{if } core(B) = t \wedge ch(B) = ch, \\ t\text{-}ch.0 & \text{otherwise,} \end{cases} \tag{8}$$

with one guard: every computed or pinned version must exceed $base_{\sigma}(P)$ in precedence, or the run refuses. Recomputing t from the *stable* baseline every run is what moves a whole train to `2.0.0-beta.0` when a breaking change lands mid-train (Table 2).

Definition 4.12 (run). A run from σ is parameterised by an adversarially chosen set $F \subseteq \Pi(\sigma)$ of failing publishes, which induces the blocked set $B = \{p \in \Pi(\sigma) \setminus F : p$ transitively depends on some $q \in F\}$, computed over the *full* workspace graph (a package with no bump this run is still a path from a dependent to a

Table 2: Eq. 8 tracing one train.

Baseline	Pending bump, channel	Next
`1.2.3`	minor, `beta`	`1.3.0-beta.0`
`1.3.0-beta.0`	no fresh commit	empty plan
`1.3.0-beta.0`	a fresh minor commit	`1.3.0-beta.1`
`1.3.0-beta.1`	minor + a new fix	`1.3.0-beta.2`
`1.3.0-beta.2`	a breaking change lands	`2.0.0-beta.0`
`1.3.0-beta.2`	none, switch to `rc`	`1.3.0-rc.0`
`1.3.0-rc.1`	none, `stable`	`1.3.0`

failed dependency). Publication proceeds in a topological order of the full graph, ties broken bytewise by name, filtered to the plan *last*: inducing the graph on the plan first deletes ordering paths that run through unplanned packages. Each package is tagged immediately after its own publish succeeds, never before (a tag for an unpublished version makes it unrecoverable, since Eq. 1 treats it as released), never batched (a mid-run failure could repeat an external write, which a target may reject or may perform twice), and *whatever its publish target*, including targets that upload nothing (Proposition 5.5). The run yields $run(\sigma, F) = \sigma \cup \{(p, v_\sigma(p), h) : p \in \Pi(\sigma) \setminus (F \cup B)\}$. A *run sequence* is $\sigma_1 \leq \sigma_2 \leq \cdots$ with $\sigma_{k+1} = run(\sigma_k, F_k)$.

For scale, let H, A count reachable commits and parent edges, C commits in the union window, U units, I resolved unit–package incidences, and Z retained or emitted contribution/provenance incidences. Window classes can share immutable reachability sets, but constructing them costs in $H + A$, not only C; stable and fresh windows require different boundary keys. Literal propagation costs $O(U(|\mathcal{P}| + |E|) + Z)$. A graph walk may be cached safely when source set, depth and edge kinds match; merging by window and directive spelling is unsafe because admission, cancellation, scope, self-source exclusion, inherited channel and provenance remain per unit. Lexical parsing may be cached by repository, immutable record, parser/schema and parsing-configuration identities. Scope and changed-path resolution has a distinct workspace/path/graph key and must be recomputed or cached separately. Channel incidence inversion costs $O(U + I + |\mathcal{P}|)$. These are informative implementation costs, not required end-to-end bounds; semantic equivalence and deterministic output, including provenance, remain normative.

Order matters for a reason tags cannot express: a dependent published before its dependency carries a range reconciled against the old version, both land at the same commit, and no later run can detect the inversion from ancestry, so it is prevented, not diagnosed. When a publish is rejected because the version already exists (a run died between publish and tag), CCME 2.0 requires content-identity verification before adoption and otherwise aborts: tagging a foreign artefact would enrol it into the release lineage. This is a protocol obligation, not a claim that dispat can inspect arbitrary user-defined publish scripts; such integrations must supply an idempotency or identity check.

Theorem 4.13 (determinism). *$\Pi(\sigma)$, v_σ, the publish order and the diagnostics are functions of $(\sigma, h, config)$ alone.*

Proof. Every definition above is a closed expression in those inputs; the three choice points (competing directives, competing propagated channels, publish-order ties) are each resolved by a stated total order: newest commit then last unit within it, and bytewise least name. □

4.4 Instantiation: What the Theorems Say About a Repository

The definitions above are parametric, and a theorem about the model is worth nothing to an operator unless it is a theorem about *their* repository. A concrete repository at a commit determines one *instance*

$$I = (\text{commits and parent edges, } h,\ \mathcal{P}, E,\ \mathcal{U},\ \sigma,\ config) :$$

the commit DAG and `HEAD` are read from the repository, the packages and edges from its manifests and configuration, the units from its commit messages, and σ from its tags. Every quantity the model defines is a computable function of I: the sets are finite, ancestry is decidable, and no definition consults anything outside I. The model is a fragment of repository behaviour: it abstracts artefact contents, hook semantics, and configuration features not represented in I. The theorems of Section 5 are universally quantified over instances of that fragment, subject to acyclicity, the version guard of Definition 4.11, and the retry invariant stated below. The latter is a property of the run sequence, checked again before each retry, rather than a static property of one instance. What remains between the theorems and a running binary is a *refinement* claim, that the implementation computes this model's $\Pi(\sigma)$ and v_σ and executes Definition 4.12's protocol. That claim is not proved here; it is tested, by Section 8.2's vectors (each fixes a small instance and the plan the model assigns it) and by Section 8.1 on randomised instances.

5 GUARANTEES, WITH PROOFS

Cross-run claims use the *retry invariant*: the corrected tuple stream, source sets, graph, scope/depth predicates, target baselines, and resolved source/target channel admission of every outstanding unit remain unchanged, except for baseline tags written for successful packages. Two recurring components deserve names. **(H1)** says every baseline channel read by `Propagate-Channel: inherit` remains unchanged; **(H2)** says no contribution initially suppressed by cancellation, hold, or correction is later released at the same h. H1 and H2 are necessary in their respective cases but are not, by themselves, the full retry invariant. Neither is needed for one invocation's determinism.

Lemma 5.1 (window antitonicity). *For $\sigma' = run(\sigma, F)$, $W^\circ_{\sigma'}(p) = \emptyset$ if p published and $W^\circ_{\sigma'}(p) = W^\circ_\sigma(p)$ otherwise. The train window W also empties after a stable release, but may remain unchanged after a prerelease release.*

Proof. The run adds only tags at h. If p gained none, both baseline commits are unchanged. If it published, $base_{\sigma'}(p) = h$, so Eq. 2 subtracts $reach(h)$ and is empty. Only a stable release also makes $scom_{\sigma'}(p) = h$. □

Lemma 5.2 (source stability). *Under the retry invariant, $src_{\sigma_j}(u)$ is constant for every outstanding unit throughout a valid run sequence.*

Proof. The corrected tuple stream and its cancellation, hold, correction, and source-set predicates are fixed components of the invariant. H2 is the component excluding later release of an initially suppressed contribution. Arbitrary tag-set inclusion outside a valid run sequence implies no such monotonicity, because a higher-version tag may point to an earlier commit. □

Lemma 5.3 (retry stability). *Under the retry invariant, an unreleased package's resolved channel, each outstanding unit's source-channel set, and $rsv(u, d)$ remain constant.*

Proof. These are precisely the channel and source-set components fixed by the retry invariant. Definition 4.9 then selects the same direct or propagated proposal, and Eq. 4 reads the same sets. □

Remark 1. *(H1) cannot be replaced by requiring stable completed releases:* `inherit` *reads $ch(base_\sigma(q))$, which the origin's own graduation moves: with* `core` *at baseline* `1.5.0-beta.0` *and commits* `feat(core)^++1: x` *and* `release(core)%stable: graduate`*, run 1 plans the consumer at* `2.0.1-beta.0` *(inherited* `beta`*) while* `core` *graduates; if the consumer fails, run 2 plans it at* `2.0.1`*: same package, both plans, different versions, every completed release stable. Thus unchanged inherited-channel state must be an explicit component of the retry invariant; successful stable releases alone do not imply version stability.*

Theorem 5.4 (the eight guarantees). *Write adm_j for adm_{σ_j} and $n = |\Pi(\sigma_1)|$. Under the retry invariant, for every unit u, consumer D and run $j \geq 1$:*

- **G1 termination** *propagation halts after $O(|\mathcal{U}| \cdot (|\mathcal{P}|+|E|))$ work, whatever the declared depths.*
- **G2 no orphans** *$adm_1(u, D) \Rightarrow adm_j(u, D)$ until D releases at a commit containing c_u.*
- **G3 version stability** *if $D \in \Pi(\sigma_1)$ and D is not released before run k, then $D \in \Pi(\sigma_k)$ and $v_k(D) = v_1(D)$.*
- **G4 no double ledger delivery** *once D releases past c_u, $adm_j(u, D)$ never holds again; with G2, ledger discharge is exactly-once for every contribution whose target eventually succeeds. This does not make external side effects exactly-once.*
- **G5 no widening** *$\{D : adm_j(u, D)\} \subseteq \{D : adm_1(u, D)\}$.*
- **G6 convergence** *$\Pi(\sigma_{n+1}) = \emptyset$, provided each nonempty retry admits and publishes at least one initially publishable, non-held package.*
- **G7 channel discharge** *a propagated channel contribution is admitted only while its commit remains in $W^\circ(D)$; baseline equality and transition matching independently remove no-op proposals.*
- **G8 one-way axis independence** *suppressing a bump never suppresses a channel change. The converse is false: resolved channels are an input to bump admission in Eq. 4.*

Proof. G1: the traversal computing R marks each package seen at most once, so it does $O(|\mathcal{P}|+|E|)$ work per unit regardless of depth or cycles; acyclicity is needed for the publish order, not here. G2: compare Eq. 5 clause by clause. Source set, depth, graph, scope, cancellation and channel admission are fixed by the retry invariant. The consumer's fresh window is unchanged until D releases (Lemma 5.1); that release is the only event that can make its admission clause false. G3: D gained no tag, so its baselines, windows and *dir* are unchanged; the propagated join is unchanged by G2 ($\geq$) and G5 ($\leq$), whose proofs below use only the lemmas; so *eff* is unchanged and the channel is unchanged (Lemma 5.3). Thus bump-only and channel-only membership remain; an exact `Release-As` remains the same because its unit is fixed by the invariant. In every case v is the same function of unchanged inputs. G4: by Lemma 5.1, $W^\circ(D)$ empties after either a stable or prerelease tag, killing fresh direct and propagated admission. This also makes $pin_\sigma(D)$ undefined. This is discharge in the tag ledger; an interruption after an external publish but before its tag is handled by content-identity reconciliation, not by an exactly-once claim about the external operation. G5: every clause of Eq. 5 is constant except the window, which is antitone (Lemma 5.1); a conjunction of constant and antitone predicates is antitone. G6: $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$, since a member of the later plan was not released (else G4) and each disjunct of Π is then unchanged or smaller; by the premise each nonempty retry removes at least one initially publishable package (G4), so $|\Pi|$ strictly decreases to 0 within n runs. A run that publishes nothing has not failed *partially* and is surfaced as a hard error, not retried. G7: the channel-axis admission test uses $c_u \in W^\circ(D)$, so Lemma 5.1 discharges it. While fresh, equality with the baseline and a transition's `<from>` check separately reject no-ops. G8: the channel pass runs first and reads no bump-axis output; the bump pass then reads its resolved channels through Eq. 4. □

Proposition 5.5 (universal tagging is necessary). *If a released package with no artefact to upload and a non-empty effective bump were versioned but not tagged, its window would never advance (Lemma 5.1), it would re-enter every plan forever, and G6 would fail. Hence Definition 4.12 tags whatever is released, and the convergence check (re-plan after a fully successful run; a non-empty result is an error) admits exactly one exclusion, held packages.*

Three corollaries carry the operational weight. *Resumption*: after a partial failure, $\Pi(\sigma_2) = F \cup B$ at the versions of run 1 (G2–G5), so the next run does the rest of the same job with no flags and no cleanup. *Idempotency*: running twice with no intervening commits, tag changes, or configuration changes gives the same plan (Theorem 4.13); running after full success gives the empty plan (G4). *Review soundness*: while the retry invariant holds, by G5 the plan a reviewer approves before the first attempt is an upper bound on everything every retry can publish, so an attacker who can induce a publish failure cannot enlarge what a commit releases, and no resume needs re-approval.

6 RELEASE-TOOL COMPARISON

In the pinned configurations we executed, Lerna and nx derive bumps from commits and tag before publish, so a failed upload is absent from their tag ledger; release-please has the same ordering in its documented workflow. Changesets tags after publish, consumes intent files, and queries npm during recovery. CCME

Table 3: Section 2 under a schedule that loses `cli` twice. G2: `cli` is in every plan until it publishes. G3: at `2.0.1` in all of them. G4: the three that published leave. G5: nothing new enters. G6: the plan reaches empty. Run 2 publishes nothing, which G6 treats as the hard-error case rather than the partial-failure one.

Run	Plan	Published	Failed
1	`core`, `cli`, `ui`, `api`	`core` 1.5.0, `ui` 0.9.2, `api` 1.2.1	`cli`
2	`cli` 2.0.1	—	`cli`
3	`cli` 2.0.1	`cli` 2.0.1	—
4	∅	—	—

Table 4: Pinned release-tool configurations.

	lerna / nx	changesets	CCME / dispat
bump source	commits	intent files	commits
tag time	before publish	after publish	after each leg
propagation	unconditional	range-gated	per commit
recovery ledger	tags / CI	npm	git tags
external gap	manual/query	npm query	reconcile identity

tags each package after its publish leg and therefore retains debt in git, subject to the external publish/tag reconciliation limit of Definition 4.12. The observed propagation policies were unconditional for Lerna/nx, range-gated for Changesets, and per-commit for CCME. Table 4 summarises the resulting operational distinctions; the artifact contains the full command transcripts. The refusal and push fixtures pin Lerna 10.0.1, Nx 23.1.2, and Changesets 3.0.1. Lerna/nx tagged the denied version before its failed upload; Changesets left it untagged and recovered by querying npm. In the concurrent push fixture, compared runs left tags outside branch ancestry, while dispat merged before its final push and planned the colleague's commit next.

7 IMPLEMENTATION

dispat is a statically linked Go binary whose release-planning state is only configuration, history, and tags. Packages are folders and lifecycle stages are shell commands; scanners and byte-preserving writers cover 35 manifest formats in twenty ecosystems. The planner implements the definitions above; topological execution continues independent subtrees after a failure and skips consumers of failed packages. A remote reserved tag serialises release runs. Successful external effects are not rolled back: later tag, record, or push failure leaves reconciliation work under the publish-then-record protocol. Invoked tools, including Terraform, retain responsibility for their operational state and identity checks. The implementation is tested through vectors, black-box CLI runs, fuzzing, real manifests, and the campaigns below.

Repository layouts and saga granularity. For a single-package repository, $\mathcal{P} = \{p\}$ and $E = \varnothing$: propagation is vacuous, but version planning, publish-then-tag recording and recovery still apply. This is the one-leg case of the saga protocol. Dispat requires the package in a subfolder; the configuration and release ledger remain at the root. Multiple destinations inside one publish script still share one completion tag, so partial external success requires reconciliation; separate package legs provide separate completion records. A single source project can expose its CLI, container image and documentation as such legs. For a polyrepo, a control repository pins linked repositories as git submodules inside package wrapper folders. Its pointer-update commits express release intent; its history and tags provide the model's single h and σ. The linked repositories retain their own histories. This supplies one release graph without merging the source repositories, rather than a distributed transaction over independent git histories. This construction establishes applicability; the evaluation does not measure polyrepo operational scale.

Reusable automation. A Windows developer and a Linux CI runner can share `global.yaml` (graph, scripts and lifecycle). Each platform file references it with `$ref: ./global.yaml`; `linux.yaml` sets `shell: ["/bin/sh", "-c"]`, while `windows.yaml` sets `["powershell.exe", "-NoProfile", "-Command"]` as its shell. Only platform overrides are repeated; references are file-relative and replace whole keys, not nested fields. Preview with `dispat --config windows.yaml status` locally or `dispat --config linux.yaml status` on the runner; CI runs the latter configuration without `status` after its release gates. `dispat if OS=Windows_NT` can select these commands through `--then` and `--else`, but 1.8.2 executes those branches through `/bin/sh` before the child reads its configuration. Native Windows can select its file directly; the conditional wrapper still needs a POSIX shell. Named scripts can themselves use `$ref`; `dispat for Debug Release --do "dispat exec build-one"` passes each value through `DISPAT_ITEM`.

Agent release gates. A lock-free `status --require-release --strict` check can precede an all-package workspace suite with verified temporary-link cleanup. Package checks then validate reconciled versions and lockfiles, followed by smoke tests of the exact artifacts before publication. Wrappers preserve failures, and release authorization remains separate. HOL Guard PR 2813 adds opt-in review of Dispat release starts [29]; Dispat and project scripts supply the test gates. On 6 September 2026 this integration proposal remained open, with its complete suite unverified. It illustrates implementation use, not deployment or another protocol guarantee.

8 EVALUATION

The primary campaign uses the official dispat 1.8.2 image and implemented CCME 2.0 semantics [8]. Release-tool comparisons retain their pinned 1.7.x measurements and are not represented as new 1.8.2 experiments. The artifact records images, seeds, raw streams, checksums, and harness versions.

8.1 Differential Validation of the Refinement Claim

Section 4.4 left one obligation open: that the binary computes this paper's model. We test it with selected specification vectors and a restricted reference oracle in approximately 300 lines of Python. The oracle predates the complete model presented here:

agreement within its core domain is evidence for that fragment, not a proof of refinement of the full model. A seeded generator produces instances with 5–20 packages, layered DAGs, 4–12 units, and mixed tag states. Each instance becomes a real Git repository and is planned by the oracle and the released binary. Golden cases gate the comparison.

The core profile isolates the oracle's shared semantic domain. The full profile deliberately exercises prerelease baselines, channel-only entries, and bump-less directive types outside that domain. Differences in the latter are classified diagnostically; they are not counted as passing conformance checks. The current campaign below reports both profiles, including rejected plans and the limits of its heuristic classification.

8.2 Released-Binary Evaluation

We reran released CCME 2.0.0 against the official dispat 1.8.2 ARM64 image. The campaign identity binds source tag commit `a7011cbcb9e8`, image digest `sha256:4c0ad4b8...e7bfd`, the verified release binary, and 753 artifacts.

The revised runner validates Git fixtures and CLI exits, uses tmpfs for E1, and distinguishes semantic refusals from execution failures. These timings are not compared with runs using an earlier harness. All 1,500 core instances agreed, with no invalid sample. Of 1,500 full-profile histories, which include regressive prereleases, 997 produced accepted plans (546 clean comparisons) and 503 produced complete E195 diagnostic refusals; four refused dossiers had no row difference, giving 550 diagnostic-clean instances overall. Every proposed regression set in the 503 dossiers was independently checked. The remaining 1,914 package-row differences retained PG1 862, PG2 375, PG2x 14, and PG3? 663, with UNKNOWN 0. Rejected diagnostic plans are neither executable release plans nor conformance results. The CLI also passed selected 2.0 vectors 133–138 (fresh discharge, suppression, channel eligibility/order, inherited-channel drift, and source exclusion). Vector 135 observes downward W208, not the upward API; six cases do not prove refinement.

Two orphan repetitions recovered the same version to an empty third plan; two campaigns converged in 24/24 episodes within four runs with zero structural G3/G5 errors. Five-release replay reproduced 36/42 google-cloud-node and 7/64 conventional-changelog versions, without implying those releases used CCME. Three fresh dispat fault cells passed 8 orphan, 12 clean-push, and 14 conflicting-push checks; competitor cells were not rerun.

All eight timing cells are valid contended-host observations under a two-CPU, 4-GiB cap, not cross-tool comparisons. Medians span 0.017 s for 100 layered packages to 19.354 s for a 10,000-package chain; observed maximum RSS spans 15–118 MiB. U1000 took 0.290 s/26 MiB and 5,000 padding commits took 0.664 s/30 MiB. The recorded fixtures passed `git fsck`, and the `commit-tree` generator matched the ordinary small fixture's commit IDs, refs, messages, and log output.

8.3 Defect-Finding Power of the Formalisation

Minimal counterexamples identify why several apparently simpler designs fail. They serve as regression obligations and explain the model's assumptions.

(1) *Aggregation is not discharge.* Reusing a stable-baseline train window for fresh admission can produce `beta.1`, `beta.2`, ... without new commits. Separate W and W° retain train intent while discharging delivered work.
(2) *Suppression must remain stable across retries.* If a cancelled or corrected source is later released, its discharged record can change the source set and reach new consumers. H2 excludes this widening within a reviewed retry sequence; changing it requires a new plan.
(3) *Inherited channels are mutable inputs.* The graduation example in Remark 1 changes a failed consumer's next version unless H1 preserves its inherited-channel inputs.
(4) *Reverse audits need the full admission predicate.* A consumer-side staleness audit that omits channel eligibility or target self-exclusion can disagree with forward propagation.
(5) *Axis independence is one-way.* Resolving channels can make a dependency ineligible and suppress a bump; suppressing a bump does not suppress a channel change. Resolving channels first preserves this order.
(6) *Shared graph traversal does not merge unit identity.* Subtracting a union of source sets excludes legitimate targets of other units; admitting every traversed node can include a unit's own source. Traversal may be shared, but admission and provenance remain per unit.

These counterexamples motivated the released rules used in the model. Their role here is to establish design necessities, not to claim defects in the current specification. The selected released-binary vectors cover six cases; they do not establish full implementation refinement.

9 THREATS TO VALIDITY

Internal. The vectors, the tests and the formal model were produced by the same author as the system, so they share assumptions and their agreement cannot catch an error common to all three. The guarantees are proved of the model, not of the Go code; the link is the vector suite, the tests and the differential run of Section 8.1, which covers the planner but not the run's protocol. The concurrent-push experiment is the strongest evidence for that protocol, and it is one deterministic interleaving per tool, not a sweep over every point a push could land. *External.* The self-hosting study is one repository of moderate size administered by the tool's author; it demonstrates feasibility and recovery on real history, not usability at organisational scale. Ecosystem generality is by construction plus tested writers and manifest fixtures, chosen by us rather than a field study. *Construct.* Conformance is measured against a contract the implementation co-evolved with; Section 8.3 shows this cuts both ways. No user study supports reviewability claims; we claim only that the blast radius is written in the message, not that authors write it correctly. The characterisation of release-please is read from its documentation; Lerna, nx and Changesets are executed on two faults in the pinned six-package fixture summarised in Section 6. The check counts recorded for those tools describe what each leaves behind against the questions dispat's protocol answers, not a score on a shared scale. The experiments run in containers on one machine and one

CI provider; the replay configurations for the subject repositories were reverse-engineered from their layouts and may misread conventions their tools resolve differently.

10 RELATED WORK

Adams and McIntosh survey release engineering as a first-class subject [20]; Potvin and Levenberg document Google's monorepo practice [21], whose build layer solves ordering and caching but not versioned publication to external registries. Among release tools, semantic-release [17] established commit-driven versioning; Lerna [15], changesets [16], Nx [18] and release-please [19] extend automation to monorepos within, predominantly, one ecosystem each; Section 6 states their computations as instances of this paper's model and locates the structural difference in the tag discipline. Empirical studies show declared version semantics and actual compatibility diverge [22, 23]; Decan et al. study dependency-network evolution across ecosystems [24]. Together they motivate explicit release intent over a dependency graph. Formal treatments of version control (semantics of VCS operations [11], patch theories as higher inductive types [12], merges as pushouts [13]) model contents and merging; this paper's model keeps only the DAG, tags and messages, the objects repository mining treats as git's model [9] and mergeable-type derivations build on [10]. Packages and dependency graphs as formal objects go back to the EDOS work on distribution consistency [14]. Within the literature surveyed here, we found no earlier composition of these objects into this release protocol. The protocol adapts sagas [1] to legs for which no acceptable compensating action is available; we therefore choose forward recovery in light of transaction limits described by Gray [2]. Helland's arguments for independently committed, idempotently reprocessable steps [3, 4] describe the discipline G2–G6 impose. Kahn's algorithm [5] underlies the publish order; Mokhov et al.'s build-system taxonomy [25] clarifies what dispat is not: it has no incremental rebuild layer, because unchanged packages are absent from the plan rather than cached. Fuzzing is reported under the cautions of Klees et al. [26]. The nearest formal-guarantees work in operations is Anvil [27], which verifies that Kubernetes controllers reconcile to a desired state; convergence here instead comes from a ledger, replay, and explicit reconciliation of uncertain external outcomes.

11 CONCLUSION

Across single-package repositories, monorepos and control-repository polyrepos, CCME treats release publication as a saga. It records intent in commit messages, completed publication legs in tags, and admits propagated work against each consumer's own fresh-work window. Under the retry invariant, its eight guarantees make deterministic replay a correct recovery procedure. Minimal counterexamples show why separate admission windows, stable retry predicates, and per-unit provenance are necessary. The pinned release-tool fault experiments support the operational distinction in the pinned configurations: dispat resumes a refused upload and preserves tag reachability across a concurrent push; the compared runs required external recovery.

Remaining work is to mechanise the model and proofs in Lean or TLA$^+$ [28] and extend differential validation from plans to complete runs, including randomised interleavings of tagging and merging.

ACKNOWLEDGMENTS: AI ASSISTANCE

OpenAI Codex agents assisted with drafting and revising the exposition and proof arguments throughout Sections 1–11, writing experiment harness code, running experiments, and summarising the resulting records. This assistance extended beyond language editing. Reported measurements were obtained from program executions rather than language-model predictions. The human author remains accountable for the claims, proofs, references, and experimental interpretation.

DATA AVAILABILITY

Released source and specifications are available at https://github.com/yohimik/dispat. A local supplementary bundle retains the proof details, vectors, fixtures, harnesses, checksums, and raw records; its archival deposit remains a publication-readiness requirement.

REFERENCES

[1] H. Garcia-Molina and K. Salem. Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.
[2] J. Gray. The transaction concept: Virtues and limitations. In *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, pages 144–154, 1981.
[3] P. Helland. Life beyond distributed transactions: an apostate's opinion. In *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007.
[4] P. Helland. Idempotence is not a medical condition. *Communications of the ACM*, 55(5):56–65, 2012.
[5] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.
[6] T. Preston-Werner. Semantic Versioning 2.0.0. https://semver.org/spec/v2.0.0.html, accessed 2026.
[7] Conventional Commits 1.0.0. https://www.conventionalcommits.org/en/v1.0.0/, accessed 2026.
[8] CCME Specification 2.0.0. https://github.com/yohimik/dispat/blob/d3ce8ae637232908ced82efb5cda550f60ed2fdf/specs/ccme-spec/SPEC.md, accessed 2026.
[9] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. Germán, and P. Devanbu. The promises and perils of mining Git. In *Proceedings of the 6th IEEE International Working Conference on Mining Software Repositories (MSR)*, 2009.
[10] G. Kaki, S. Priya, K. C. Sivaramakrishnan, and S. Jagannathan. Mergeable replicated data types. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA), Article 154, 2019.
[11] W. Swierstra and A. Löh. The semantics of version control. In *Proceedings of the ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward!)*, pages 43–54, 2014.
[12] C. Angiuli, E. Morehouse, D. R. Licata, and R. Harper. Homotopical patch theory. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 2014.
[13] S. Mimram and C. Di Giusto. A categorical theory of patches. *Electronic Notes in Theoretical Computer Science*, 298, 2013.
[14] F. Mancinelli, J. Boender, R. Di Cosmo, J. Vouillon, B. Durak, X. Leroy, and R. Treinen. Managing the complexity of large free and open source package-based software distributions. In *Proceedings of the 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2006.
[15] Lerna: a fast, modern build system for managing and publishing multiple JavaScript packages from the same repository. https://lerna.js.org/, accessed 2026.
[16] Changesets: a way to manage your versioning and changelogs. https://github.com/changesets/changesets, accessed 2026.
[17] semantic-release: fully automated version management and package publishing. https://github.com/semantic-release/semantic-release, accessed 2026.
[18] Nx: smart monorepos, fast CI. https://nx.dev/, accessed 2026.
[19] release-please: automated releases based on conventional commits. https://github.com/googleapis/release-please, accessed 2026.

[20] B. Adams and S. McIntosh. Modern release engineering in a nutshell: Why researchers should care. In *Proceedings of the 23rd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, 2016.

[21] R. Potvin and J. Levenberg. Why Google stores billions of lines of code in a single repository. *Communications of the ACM*, 59(7):78–87, 2016.

[22] S. Raemaekers, A. van Deursen, and J. Visser. Semantic versioning versus breaking changes: A study of the Maven repository. In *Proceedings of the 14th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2014.

[23] C. Bogart, C. Kästner, J. Herbsleb, and F. Thung. How to break an API: Cost negotiation and community values in three software ecosystems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, 2016.

[24] A. Decan, T. Mens, and P. Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019.

[25] A. Mokhov, N. Mitchell, and S. Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, 2018.

[26] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks. Evaluating fuzz testing. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2123–2138, 2018.

[27] X. Sun, W. Ma, J. T. Gu, Z. Ma, T. Chajed, J. Howell, A. Lattuada, O. Padon, L. Suresh, A. Szekeres, and T. Xu. Anvil: Verifying liveness of cluster management controllers. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[28] L. Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[29] Semyon F. (yohimik). Opt-in Dispat release-start review. *hashgraph-online/hol-guard*, pull request 2813, 2026. https://github.com/hashgraph-online/hol-guard/pull/2813. Head `132bb5d105e1`; accessed 6 September 2026.